Lecture 3

Assembly Language Fundamentals — Lecture Notes

I. **Basic Elements.** Constants, integer expressions, reserved words, identifiers, directives, instructions, labels, operands and comments.

II. **A First Program.** AddSub, line by line, and the program template.

III. **Defining Data.** `BYTE` … `REAL10`, arrays, strings, `DUP`, little-endian order, uninitialized data.

IV. **Symbolic Constants.** `=`, `$` and array sizes, `EQU`, `TEXTEQU`.

V. **Exercises & Assignments.** Practice questions from the slides; no solutions included.

Notation. [S39] means slide 39 of Lecture 3.

Mahmoud AboZied
Teaching Assistant

Academic Year 2026 / 2027

Contents

Part I · Basic Elements of Assembly Language 3
1 · Integer Constants [S4] 3
2 · Integer Expressions [S5, 6] 3
3 · Real Number Constants [S7] 3
4 · Character and String Constants [S8] 3
5 · Reserved Words [S9] 4
6 · Identifiers [S10, 11] 4
7 · Directives [S13, 14, 15] 4
8 · Instructions [S16, 17] 5
9 · Labels [S18, 19] 5
10 · Mnemonics and Operands [S20, 21, 22] 5
11 · Comments [S23] 5

Part II · A First Program 7
12 · AddSub — Adding and Subtracting Integers [S24, 29] 7
13 · Program Template [S30, 56] 7
14 · AddSub2 — Storing the Sum in a Variable [S55, 57, 69] 8

Part III · Defining Data 9
15 · Data Definition Directives [S32, 33] 9
16 · The Data Definition Statement [S34] 9
17 · BYTE and SBYTE [S35] 9
18 · Byte Arrays — Multiple Initializers [S37] 9
19 · Strings [S40, 42, 43, 44] 10
20 · The DUP Operator [S46] 10
21 · WORD, DWORD, QWORD, TBYTE, Real [S50, 51, 52] 11
22 · Little-Endian Order [S53, 54] 11
23 · Uninitialized Data [S58] 11

Part IV · Symbolic Constants 12
24 · The Equal-Sign Directive [S60, 61] 12
25 · Calculating the Size of an Array — $ [S62, 63, 65] 12
26 · EQU [S66, 67] 12
27 · TEXTEQU [S68] 12

Part V · Exercises & Assignments 14
Exercise 1 — Identifiers [S12] 14
Exercise 2 — BYTE Declarations [S36] 14
Exercise 3 — Array elements [S38] 14
Exercise 4 — An array across three lines [S39] 14
Exercise 5 — A string declaration [S41] 14
Exercise 6 — Two-line string [S45] 14
Exercise 7 — DUP [S47] 15
Exercise 8 — Multiple DUP expressions [S48] 15
Exercise 9 — Little-endian [S54] 15
Exercise 10 — Sizes of arrays and strings [S64, 65] 15
Exercise 11 — EQU [S66, 67] 15
Exercise 12 — C to MASM [S70] 15
Assignment 1 [S49] 15
Assignment 2 [S71] 16

Part I · Basic Elements of Assembly Language

The building blocks covered in this part: integer constants · integer expressions · character and string constants · reserved words and identifiers · directives and instructions · labels · mnemonics and operands · comments. [S3]

1 · Integer Constants [S4]

```
[{+ | -}] digits [radix]
```

Radix	Meaning	Examples
`h`	hexadecimal	`0A3h`, `19h`
`d`	decimal	`26`, `26d`
`b`	binary	`0101010b`, `11001b`
`q` / `o`	octal	`42q`, `42o`, `31o`

- Optional leading `+` or `-` sign.
- **No radix → decimal.**
- A hex constant that **begins with a letter** needs a leading **0**: `0A5h`, not `A5h` (otherwise MASM reads it as a name).

```
mov eax, 25      ; decimal
mov eax, 19h     ; hexadecimal
mov eax, 11001b  ; binary
mov eax, 31o     ; octal
mov eax, -10     ; decimal
```

The first four `mov` examples load the **same value**, 25: `19h` = `11001b` = `31o` = 25.

2 · Integer Expressions [S5, 6]

Operator	Name	Level
`( )`	parentheses	1 (highest)
`+`, `-`	unary plus, minus	2
`*`, `/`	multiply, divide	3
`MOD`	modulus	3
`+`, `-`	add, subtract	4 (lowest)

Expression	Value
`16 / 5`	3 *(integer division)*
`-(3 + 4) * (6 - 1)`	−35
`-3 + 4 * 6 - 1`	20
`25 mod 3`	1

Expression	Order of evaluation
`4 + 5 * 2`	multiply, then add ⟶ 14
`12 - 1 MOD 5`	modulus, then subtract ⟶ 12 − 1 = 11
`-5 + 2`	unary minus, then add ⟶ −3
`(4 + 2) * 6`	add, then multiply ⟶ 36

3 · Real Number Constants [S7]

A **decimal real**: optional sign, integer, decimal point, optional fraction, optional exponent.

`[sign] integer.[integer] [exponent]` sign = `{+, -}` exponent = `E[{+, -}] integer`

Examples: `2.` `+3.0` `-44.2E+05`

4 · Character and String Constants [S8]

- Enclose a **character** in single or double quotes: `'A'`, `"x"`. One ASCII character = **1 byte**.

- Enclose a **string** in single or double quotes: `"ABC"` (3 bytes), `'xyz'` (3 bytes). Each character occupies one byte.
- **Embedded quotes**: `'Say "Goodnight," Gorg'`. The single quotes delimit the string, and the double quotes inside are ordinary characters. The string is: Say "Goodnight," Gorg.

5 · Reserved Words [S9]

Reserved words **cannot be used as identifiers**. They include:

Kind	Meaning	Examples
Instruction mnemonics	The names of CPU instructions	`MOV`, `ADD`, `MUL`
Register names		`EAX`, `EBX`, `AX`
Directives	Tell MASM how to assemble the program (e.g. reserve space for variables)	`.data`, `PROC`
Type attributes	Give size and usage information for variables and operands	`BYTE`, `WORD`
Operators	Used in constant expressions	`+`, `MOD`
Predefined symbols	Return constant integer values	`@data`

6 · Identifiers [S10, 11]

A **programmer-chosen name** for a variable, constant, procedure or code label.

Rules
- **1 – 247 characters**, digits included.
- **Not case-sensitive.**
- First character: a **letter** (A–Z, a–z), `_`, `@`, `?` or `$`.
- Cannot be a **reserved word**.
- Avoid a leading `@`: the assembler uses it for predefined symbols.

Valid: `Var1`, `Count`, `MAX`, `open_file`, `myFile`, `xVal`, `NUMBER?`, `NUMBER$`

Invalid	Why
`2VALUE`, `123`, `123ABC`	Starts with a digit
`MY-VAR`	`-` is not allowed
`MY VAR`	Spaces are not allowed
`MOV`, `ADD`	Reserved mnemonic
`AX`, `BX`	Register name (reserved)
`A+B`	`+` is an operator
`VALUE=`	`=` is not allowed
`;TEST;`	`;` starts a comment

7 · Directives [S13, 14, 15]

A **directive** is a command recognized and acted upon by the **assembler**.
- **Not** part of the Intel instruction set: it does not become a machine instruction.
- Used to declare code and data areas, select the memory model, declare procedures, define variables and macros.
- **Not case-sensitive** (`.data` = `.DATA` = `.Data`). Different assemblers have different directives.

Directive	Example	Meaning
`DB`	`NUM DB 25`	Define Byte (1 byte)
`DW`	`NUM DW 1234H`	Define Word (2 bytes)
`DD`	`NUM DD 12345678H`	Define Doubleword (4 bytes)
`DUP`	`ARR DB 5 DUP(0)`	**Duplicate**: 5 bytes, all initialized to 0
`EQU`	`MAX EQU 100`	Define a **constant**
`.DATA`	`.data`	Begins the **data** segment (variables)
`.CODE`	`.code`	Begins the **code** segment (executable instructions)
`.STACK`	`.stack 100h`	The runtime **stack** segment and its size
`END`	`END main`	Marks the end of the program and (optionally) the entry point

8 · Instructions [S16, 17]

An **instruction** is a statement that becomes **executable** when the program is assembled: the assembler turns it into machine code and the CPU executes it at run time.

Syntax.

```
[label:]  mnemonic  [operands]  [; comment]
```

Label	Mnemonic	Operands	Comment
optional	**required**	depend on the instruction	optional

Example: `mov eax, 10 ; Move 10 into EAX` ⟶ mnemonic `mov`, operands `eax, 10`, comment `; Move 10 into EAX`.

9 · Labels [S18, 19]

A label is a **place marker**: it marks the address of data or code, and follows the identifier rules.

Data label — no colon

- Identifies the location of a variable so code can reference it. Must be unique.

```
count  DWORD 100
number DWORD 100   ; store 100 in memory
```

`number` ⟶ label · `DWORD` ⟶ directive · `100` ⟶ value.

Code label — followed by a colon

- The target of jump and loop instructions.

```
target:
    mov ax, bx
    ...
    jmp target
subtract: sub eax, 5  ; subtract 5 from EAX
```

10 · Mnemonics and Operands [S20, 21, 22]

Mnemonic	Meaning
`mov`	Move (assign) one value to another
`add`	Add two values
`sub`	Subtract one value from another
`mul`	Multiply two values
`inc`	Add 1
`jmp`	Jump to a new location
`call`	Call a procedure

Operands are the data an instruction works on:

Type	Example
Constant	`96`
Constant expression	`2 + 4`
Register	`eax`
Memory (data label)	`count`

Constants and constant expressions are called **immediate values**.

Instruction formats [S22]

```
stc                ; no operands     — set the Carry flag
inc  eax           ; one operand     — register
inc  myByte        ; one operand     — memory
add  ebx, ecx      ; two operands    — register, register
sub  myByte, 25    ; two operands    — memory, constant
add  eax, 36 * 25  ; two operands    — register, constant expression
imul eax, ebx, 5   ; three operands  — EAX = EBX × 5
```

11 · Comments [S23]

Single-line: begin with a **semicolon** `;`.

```
inc eax   ; add 1 to EAX
```

Multi-line: the `COMMENT` directive and a programmer-chosen character; the comment ends at the same character.

```
COMMENT !
   This line is a comment
   This line is also a comment
!
```

Part II · A First Program

12 · AddSub — Adding and Subtracting Integers [S24, 29]

```
TITLE Add and Subtract     (AddSub.asm)
; This program adds and subtracts 32-bit integers.
INCLUDE Irvine32.inc

.code
main PROC
    mov  eax,10000h   ; EAX = 10000h
    add  eax,40000h   ; EAX = 50000h
    sub  eax,20000h   ; EAX = 30000h
    call DumpRegs     ; display registers
    exit
main ENDP
END main
```

Output (registers and flags):

```
EAX=00030000  EBX=7FFDF000
ECX=00000101  EDX=FFFFFFFF
ESI=00000000  EDI=00000000
EBP=0012FFF0  ESP=0012FFC4
EIP=00401024  EFL=00000206
CF=0  SF=0  ZF=0  OF=0
```

EAX = **30000h**, as expected.

Line by line *[S25, 26, 27, 28]*

Line	Meaning
`TITLE …`	Marks the whole line as a comment. You can put anything on it.
`; …`	Everything right of a semicolon is ignored by the assembler.
`INCLUDE Irvine32.inc`	Copies definitions and setup information from the file `Irvine32.inc`.
`.code`	Begins the **code segment**, where all executable statements are.
`main PROC`	`PROC` marks the **beginning of a procedure**.
`mov eax,10000h`	Moves (copies) 10000h into EAX.
`add eax,40000h`	Adds 40000h to EAX.
`sub eax,20000h`	Subtracts 20000h from EAX.
`call DumpRegs`	Calls a procedure that displays the CPU registers.
`exit`	A **macro** (defined in Irvine32.inc) that indirectly calls a Windows function to halt the program.
`main ENDP`	`ENDP` marks the **end of the procedure**.
`END main`	`END` marks the **last line** to be assembled. `main` is the entry point.

13 · Program Template [S30, 56]

```
TITLE Program Template          (Template.asm)
;*************************************************
; Program Name / Description / Author / Version / Date
;*************************************************
INCLUDE Irvine32.inc
.data
    ; (insert variables here)
.code
main PROC
    ; (insert executable instructions here)
    exit
main ENDP
    ; (insert additional procedures here)
END main
```

14 · AddSub2 — Storing the Sum in a Variable [S55, 57, 69]

```
TITLE Add and Subtract, Version 2 (AddSub2.asm)
INCLUDE Irvine32.inc
.data
val1     DWORD 10000h
val2     DWORD 40000h
val3     DWORD 20000h
finalVal DWORD ?
.code
main PROC
    mov  eax,val1       ; start with 10000h
    add  eax,val2       ; add 40000h
    sub  eax,val3       ; subtract 20000h
    mov  finalVal,eax   ; store 30000h
    call DumpRegs
    exit
main ENDP
END main
```

- The values now live in **variables** in the `.data` segment.
- `finalVal DWORD ?` reserves 4 bytes with no initial value. The result is stored there.
- **16-bit version** (`Irvine16.inc`, real mode): the program must first point **DS** at the data segment:

```
mov ax,@data   ; initialize DS
mov ds,ax
```

`@data` is a predefined symbol. A segment register can't be loaded with an immediate, hence the trip through AX.

Part III · Defining Data

15 · Data Definition Directives [S32, 33]

Legacy	Defines
`DB`	byte (8-bit)
`DW`	word (16-bit)
`DD`	doubleword (32-bit)
`DQ`	quadword (64-bit)
`DT`	ten bytes (80-bit)

Directive	Bytes	Defines
`BYTE` / `SBYTE`	1	8-bit unsigned / **signed** integer
`WORD` / `SWORD`	2	16-bit unsigned / signed integer
`DWORD` / `SDWORD`	4	32-bit unsigned / signed integer
`FWORD`	6	48-bit integer
`QWORD`	8	64-bit integer
`TBYTE`	10	80-bit integer
`REAL4`	4	32-bit short real
`REAL8`	8	64-bit long real
`REAL10`	10	80-bit extended real

*The `S` prefix means **signed**. Slide 33 calls REAL8 a "short real"; it is the 64-bit **long** real.*

16 · The Data Definition Statement [S34]

```
[name]  directive  initializer [, initializer] ...
```

- All initializers become **binary data** in memory.
- The **name** (a data label) is optional.
- Use `?` for an **uninitialized** value.

17 · BYTE and SBYTE [S35]

```
value1 BYTE  'A'    ; character constant
value2 BYTE  0      ; smallest unsigned byte
value3 BYTE  255    ; largest unsigned byte
value4 SBYTE -128   ; smallest signed byte
value5 SBYTE +127   ; largest signed byte
value6 BYTE  ?      ; uninitialized byte
```

- MASM does **not** prevent initializing a `BYTE` with a negative value, but it is poor style.
- A `SBYTE` variable is shown by the debugger in decimal **with a sign**.

18 · Byte Arrays — Multiple Initializers [S37]

```
list1 BYTE 10,20,30,40
list2 BYTE 10,20,30,40
      BYTE 50,60,70,80
      BYTE 81,82,83
list3 BYTE ?,32,41h,00100010b
list4 BYTE 0Ah,20h,12h,22h
```

Label	Offset	Value
list1	0000	10
	0001	20
	0002	30
	0003	40
list2	0004	10
	0005	20
	0006	30
	0007	40
	0008	50
	0009	60

- Variables are laid out **one after another** in memory.
- A line without a name continues the previous variable: `list2` holds **11** bytes.

- Initializers can use **different radixes** in one list.

Label	Offset	Value
	000A	70
	000B	80
	000C	81
	000D	82
	000E	83
list3	000F	?
	0010	32
	0011	41h
	0012	22h

Note: the table on slide 37 shows list3 at offset 0010 with value 84. `list2` *ends at 000E, so* `list3` *starts at* ***000F****, and its first value is* `?`*.*

19 · Strings [S40, 42, 43, 44]

A **string** is an array of characters, usually in quotes and often **null-terminated** (ending with `,0`).

```
str1     BYTE "Enter your name",0
str2     BYTE 'Error: halting program',0
str3     BYTE 'A','E','I','O','U'
greeting BYTE "Welcome to the Encryption Demo program "
         BYTE "created by Kip Irvine.",0
```

To continue one string across lines, **end each line with a comma**:

```
menu BYTE "Checking Account",0dh,0ah,0dh,0ah,
   "1. Create a new account",0dh,0ah,
   "2. Open an existing account",0dh,0ah,
   "3. Credit the account",0dh,0ah,
   "4. Debit the account",0dh,0ah,
   "5. Exit",0ah,0ah,
   "Choice> ",0
```

On screen:

```
Checking Account

1. Create a new account
2. Open an existing account
3. Credit the account
4. Debit the account
5. Exit

Choice>
```

0Dh, 0Ah = new line. **0Dh** (13, **CR**, carriage return) moves the cursor to the **start of the current line**. **0Ah** (10, **LF**, line feed) moves it to the **next line**. Together they start a new line.

20 · The DUP Operator [S46]

DUP = duplicate. It allocates storage for many items, using a constant expression as a counter.

```
counter DUP ( argument )
```

Declaration	Size	Contents
`var1 BYTE 20 DUP(0)`	20 bytes	twenty zeros
`var2 BYTE 20 DUP(?)`	20 bytes	uninitialized
`var3 BYTE 4 DUP("STACK")`	20 bytes	“STACKSTACKSTACKSTACK”
`var4 BYTE 10, 3 DUP(0), 20`	5 bytes	10, 0, 0, 0, 20
`var5 BYTE 3 DUP(0,1)`	6 bytes	0, 1, 0, 1, 0, 1

Size rule. Size = (number of values) × (bytes per value). `n DUP(list)` gives **n × (length of list)** values.

21 · WORD, DWORD, QWORD, TBYTE, Real [S50, 51, 52]

```
word1 WORD  65535      ; largest unsigned
word2 SWORD -32768     ; smallest signed
word3 WORD  ?          ; uninitialized
list  WORD  1,2,3,4,5  ; array of words
array WORD  5 DUP(?)   ; 5 words, uninit.
val1  DW    65535      ; legacy DW
```

```
val1 DWORD  12345678h     ; unsigned
val2 SDWORD -2147483648   ; signed
val3 DWORD  20 DUP(?)     ; array
val4 SDWORD -3,-2,-1,0,1  ; signed array
val5 DD     12345678h     ; legacy DD
```

```
quad1      QWORD 1234567812345678h
val1       TBYTE 100000000123456789Ah
rVal1      REAL4 -2.1
ShortArray REAL4 20 DUP(0.0)
```

22 · Little-Endian Order [S53, 54]

Endianness is how the bytes of a value are ordered in memory. x86 is **little-endian**: every type larger than a byte is stored with its **least-significant byte at the lowest address**.

`val1 DWORD 12345678h`	Little endian	Big endian
`val1 + 0` (low)	**78**	12
`val1 + 1`	**56**	34
`val1 + 2`	**34**	56
`val1 + 3` (high)	**12**	78

Little-endian: **78 56 34 12**.
Some other systems use **big-endian** order (most significant byte first).

23 · Uninitialized Data [S58]

Declare uninitialized data with `?`. Memory is reserved only when the program runs, so the **EXE file is smaller**.[1]

```
.data?
bigArray   DWORD 5000 DUP(?)   ; 20,000 bytes
smallArray DWORD 10 DUP(?)     ; 40 bytes
array      BYTE  20 DUP(?)     ; 20 bytes
var1       BYTE  ?             ; 1 byte
```

[1] Slide 58 says ".data". The directive for an uninitialized data segment is actually `.data?`.

Part IV · Symbolic Constants

A **symbolic constant** is a name for a value. It is handled by the assembler and uses **no memory**. Four tools: `=`, `$` (array sizes), `EQU`, `TEXTEQU`. [S59]

24 · The Equal-Sign Directive [S60, 61]

```
name = expression
```

- `expression` is a **32-bit integer** (constant or expression). `C = 3.14` is an **error** (not an integer).
- **No run-time impact**: not part of the `.exe`. The assembler replaces the name with its value.
- Good programming style: `COUNT = 500` … `mov al, COUNT`.

> **`=` vs. `BYTE`.** `COUNT = 10` ⟶ a **constant**, not a memory location.
> `COUNT BYTE 10` ⟶ a **variable**: one byte of memory initialized to 10.

25 · Calculating the Size of an Array — `$` [S62, 63, 65]

`$` is the **current location counter**: the offset of the current statement. Written **immediately after** an array, `$ - list` = the number of **bytes** in the array.

Declaration	Size formula	Result
`list BYTE 10,20,30,40`	`ListSize = ($ - list)`	4
`myString BYTE "This is a long string, containing"`	`myString_len = ($ - myString)`	33
`list WORD 1000h,2000h,3000h,4000h`	`ListSize = ($ - list) / 2`	8 / 2 = 4
`list DWORD 10000000h,20000000h`	`ListSize = ($ - list) / 4`	8 / 4 = 2

Divide by the element size — **1** (BYTE), **2** (WORD), **4** (DWORD) — to get the **number of elements**.

26 · EQU [S66, 67]

Defines a symbol as an **integer** or a **text** expression. **Cannot be redefined.**

```
name EQU expression
name EQU symbol
name EQU <text>

PI       EQU <3.1416>
pressKey EQU <"Press any key to continue...",0>
.data
prompt BYTE pressKey     ; same as: prompt BYTE "Press any key to continue...",0
```

Declaration	Meaning
`matrix1 EQU 10 * 10`	Evaluated **now**: `matrix1` = the integer **100**.
`matrix2 EQU <10 * 10>`	Stored as **text**: wherever `matrix2` appears, the text `10 * 10` is pasted in.

27 · TEXTEQU [S68]

Defines a symbol as text, a text macro, or (with `%`) the text of an evaluated expression. **Can be redefined.**

```
name TEXTEQU <text>
name TEXTEQU textmacro
name TEXTEQU %constExpr          ; evaluate, then convert to text

continueMsg TEXTEQU <"Do you wish to continue (Y/N)?">
rowSize = 5
.data
prompt1 BYTE continueMsg
count   TEXTEQU %(rowSize * 2)   ; "10"
move    TEXTEQU <mov>
setupAL TEXTEQU <move al,count>  ; "mov al,10"
.code
setupAL                          ; assembles as: mov al,10
```

	`=`	`EQU`	`TEXTEQU`
Value type	Integer only	Integer or text	Text (or `%` expression)
Redefine?	**Yes**[2]	**No**	**Yes**
Uses memory?	No	No	No

[2] Slide 60 says "Can not redefine". In MASM a `=` symbol **can** be redefined later in the source; `EQU` cannot.

Part V · Exercises & Assignments

Exercise 1 — Identifiers [S12]

For each identifier, state whether it is valid in MASM and explain your reasoning: `NUMBER1`, `2NUMBER`, `MY_VAR`, `MY-VAR`, `COUNT`, `ADD`.

Exercise 2 — BYTE Declarations [S36]

Explain what each declaration allocates and stores. State its size in bytes.

```
value1 BYTE 'A'
value2 BYTE 0
value3 BYTE 255
value4 SBYTE -128
value5 SBYTE +127
value6 BYTE ?
```

Exercise 3 — Array elements [S38]

```
list3 BYTE ?,32,41h,00100010b
```

How many elements and bytes are allocated? Convert each initialized element to decimal and explain the meaning of `?`.

Exercise 4 — An array across three lines [S39]

```
list2 BYTE 10,20,30,40
      BYTE 50,60,70,80
      BYTE 81,82,83
```

How many elements and bytes are allocated? What are the first and last elements?

Exercise 5 — A string declaration [S41]

```
str1 BYTE "Enter your name",0
```

Identify the label and element size. Count the text characters and total bytes, explain the final `0`, and describe the order of the bytes in memory.

Exercise 6 — Two-line string [S45]

```
str1    BYTE "Enter your name:    ",0Dh,0Ah
        BYTE "Enter your address: ",0
newLine BYTE 0Dh,0Ah,0
```

Describe the output produced by each string. Explain the roles of `0Dh`, `0Ah`, and `0`.

Exercise 7 — DUP [S47]

```
var4 BYTE 10, 3 DUP(0), 20
```

Expand the declaration into its individual values and calculate its size in bytes.

Exercise 8 — Multiple DUP expressions [S48]

```
var5 BYTE 5, 2 DUP(10), 3, 4 DUP(0)
```

Expand the declaration into its individual values and calculate its size in bytes.

Exercise 9 — Little-endian [S54]

```
value DWORD 12345678h
```

Write the bytes in memory from the lowest address to the highest address.

Exercise 10 — Sizes of arrays and strings [S64, 65]

For each declaration, calculate the size in bytes and the number of elements. Write an expression using `$` to compute each quantity immediately after its declaration.

```
list1 BYTE 10,20,30,40
myString BYTE "This is a long string, containing"
list2 WORD 1000h,2000h,3000h,4000h
list3 DWORD 10000000h,20000000h,30000000h,40000000h
```

Exercise 11 — EQU [S66, 67]

```
pressKey EQU <"Press any key to continue...",0>
matrix1  EQU 10 * 10
matrix2  EQU <10 * 10>
.data
prompt BYTE pressKey
M1     WORD matrix1
M2     WORD matrix2
```

Explain the type and substitution of each symbolic constant. Determine the size of `prompt`, the values stored in `M1` and `M2`, and when each expression is evaluated.

Exercise 12 — C to MASM [S70]

Write an assembly program for `X = (a + b) * 3`. Declare suitable variables and show how to inspect the stored result.

Assignment 1 [S49]

```
var5 BYTE 3, 2 DUP(1,2), 3, 3 DUP(2 DUP(0))
```

How many bytes does `var5` occupy? What values are stored? What do `2 DUP(1,2)` and `3 DUP(2 DUP(0))` mean?

Assignment 2 [S71]

Write two assembly programs for the C statements `X = (a + b) / 2` and `a = a * b / c`.

— End of Lecture 3 —